CSED233 · 데이터구조 · 포스텍

정리본 7. Heaps

Priority Queue의 동기 · Heap의 두 성질 · Insert/DeleteMin · Heapify가 O(n)인 이유.

1. Priority Queue(우선순위 큐)

일반 큐는 먼저 온 것부터 처리한다(FIFO). 그런데 응급실에서 응급(3) > 중증(2) > 일반(1)처럼 우선순위가 다르면? 일반 큐는 [일반, 응급, 중증] 순으로 들어와도 일반부터 처리한다 — 우선순위를 무시한다.

Priority Queue(우선순위 큐): 들어온 순서가 아니라 우선순위 순으로 꺼내는 큐.

- `Insert(x)` : 우선순위를 유지하며 삽입.
- `DeleteMin()` : 최소(또는 최대) 우선순위 원소 추출.

예: Insert(10, 5, 8, 3) → DeleteMin()을 반복하면 3, 5, 8, 10 순으로 자동 정렬되어 나온다.

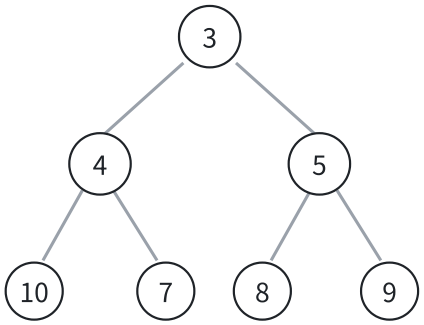
1.1 구현 방법에 따른 성능 — Heap이 "세 번째 정답"

구현	Insert	DeleteMin	병목
정렬 안 된 리스트	$O(1)$	$O(n)$ (최솟값 탐색)	DeleteMin
정렬된 리스트	$O(n)$	$O(1)$	Insert
Heap	$O(\log n)$	$O(\log n)$	**없음 — 둘 다 빠름**

2. Heap의 두 가지 성질

Heap의 조건

1. **Structural Property(구조 조건)**: Complete Binary Tree — 마지막 레벨 전까지 가득, 마지막은 왼쪽부터. 그래서 배열로 구현 가능.
2. **Heap Order Property(순서 조건)**: Min-Heap은 부모 ≤ 자식(루트 = 최소), Max-Heap은 부모 ≥ 자식(루트 = 최대).



3	4	5	10	7	8	9
[1]	[2]	[3]	[4]	[5]	[6]	[7]

Min-Heap 확인: 3≤4 ✓ 3≤5 ✓ 4≤10 ✓ 4≤7 ✓ 5≤8 ✓ 5≤9 ✓
배열 그대로: [3, 4, 5, 10, 7, 8, 9]

완전 이진 트리 모양이므로 배열과 1:1 대응

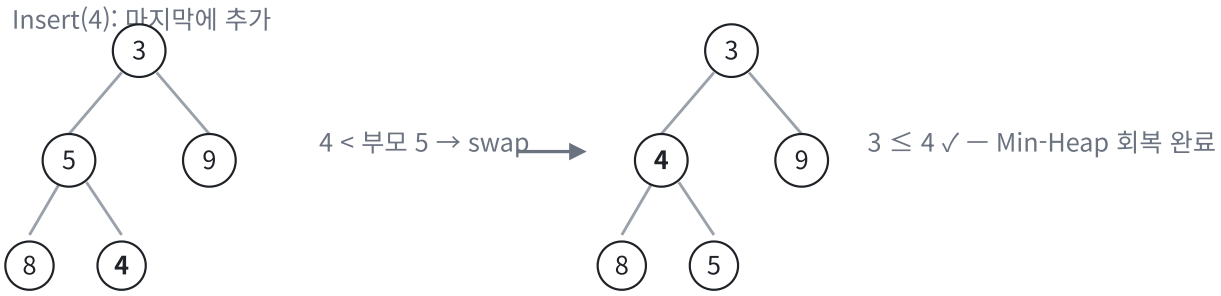
3. 기본 연산: Insert와 DeleteMin

3.1 Insert — Up-heap Bubbling, $O(\log n)$

① 새 원소를 **마지막 위치**(다음 빈 칸)에 넣는다 — Complete 형태 유지.

② 부모와 비교해 **부모보다 작으면 swap** — 위로 거슬러 올라간다.

③ 부모 ≤ 자식이 되면 멈춘다. 최악 = 트리 높이.



3.2 DeleteMin — Down-heap Bubbling, $O(\log n)$

① 루트(최솟값)를 꺼내고, **마지막 노드를 루트로** 옮긴다.

② 두 자식 중 **더 작은 쪽**과 비교해 자식이 더 작으면 swap — 아래로 내려간다.

③ 부모 ≤ 자식이 되면 멈춘다.

예제 · DeleteMin

```
   3        8        4        4
  / \      / \      / \      / \
 4   9  →  4   9  →  8   9  →  5   9
/ \      /       /        /
5   8    5       5        8
(1) 루트 삭제   (2) 8 > 4 → swap   (3) 8 > 5 → swap
    마지막 8을 루트로
```

배열: [3,4,9,5,8] → 3 삭제·8을 루트로 [8,4,9,5] → swap(8,4) [4,8,9,5] → swap(8,5) [4,5,9,8]

> **Down-heap에서 비교 상대**
>
> 자식 둘 중 **아무하고나** 비교하면 안 된다. 반드시 **더 작은 자식**과 비교해야 swap 후에도 Min-Heap 성질(부모 ≤ 두 자식 모두)이 보장된다.

4. Heap Sort(힙 정렬)

---

① 배열을 Max-Heap으로 만든다 → $O(n)$ (아래 Heapify).

② 루트(최댓값)를 n번 추출하며 뒤에서부터 채운다 → $O(n \log n)$.

총 $O(n \log n)$, 공간은 $O(1)$ (in-place, 배열 내부 정렬·반복식 Down-heap 기준).

5. Heapify — 힙을 O(n)에 만드는 법

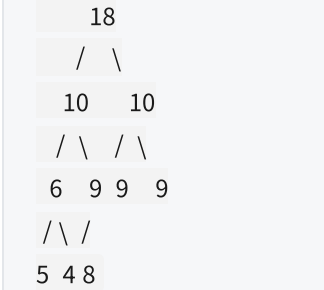
방법	과정	시간
① 순차 삽입	빈 힙에 n번 Insert	$O(n \log n)$ (최악)
② **Heapify**	원소를 전부 배열에 넣은 뒤, **마지막 비단말 노드($\lfloor n/2 \rfloor$)부터 루트까지 역순**으로 Down-heap	$O(n)$

왜 끝에서부터인가: Down-heap은 자식 서브트리가 이미 힙이어야 부모를 끌어내릴 수 있다. 리프는 그 자체로 힙이므로, 가장 작은 서브트리부터 Bottom-up으로 합쳐 올라간다.

예제 · Min-Heap → Max-Heap 변환 (n=10)

초기: [4, 5, 9, 6, 8, 9, 10, 10, 18, 9] (1-indexed)
인덱스 5(=⌊10/2⌋) → 4 → 3 → 2 → 1 순서로 Down-heap

결과: [18, 10, 10, 6, 9, 9, 9, 5, 4, 8]



왜 Heapify는 O(n)인가

- 레벨 k에는 최대 2^k개의 노드, 각 노드의 Down-heap 최대 이동 거리는 (h − k). j = h−k로 치환하고 $2^h \le n$을 쓰면 높이 j인 노드는 최대 $n/2^{j+1}$개 꼴이 된다.
- 트리 아래쪽(높이 작은 쪽)에 노드가 몰려 있어 대부분의 노드는 조금만 이동한다.

$$\sum_{k=0}^{h} 2^k(h-k) \;\le\; n\sum_{j=0}^{h}\frac{j}{2^j} \;\le\; 2n \;=\; O(n)$$

한 장의 그림으로 말하면: Insert n번은 "모든 노드가 루트까지 오를 수 있는" 비용을 치르지만, Heapify는 절반은 0번, 1/4은 1번, 1/8은 2번만 이동한다.

6. Quiz

Q1. Heap이 정렬된 리스트보다 나은 이유는?

힌트

섹션 1.1 표: 정렬된 리스트는 한쪽 연산이 O(n)이다.

정답

개념은 힌트 참고.

정답: 정렬된 리스트는 Insert O(n) + DeleteMin O(1)이라 Insert가 병목. Heap은 둘 다 O(log n)이라 전체 효율이 낫다.

Q2. Min-Heap에서 두 번째로 작은 값은 어디 있나?

힌트

Heap order는 "부모 ≤ 자식"만 보장한다. 형제끼리의 순서는?

정답

개념은 힌트 참고.

정답: 서로 다른 키를 갖는 Min-Heap에서 두 번째로 작은 값은 루트의 두 자식 중 작은 쪽에 있다. 다만 어느 쪽인지는 사전에 정해져 있지 않다 (Heap은 부분 정렬).

Q3. Heap이 배열로 구현되는 이유는?

힌트

정리본 5의 Complete Binary Tree 배열 구현과 같은 원리.

> **정답**

개념은 힌트 참고.

정답: Complete Binary Tree라 빈 공간 없이 배열에 차곡차곡 들어가고, 인덱스 계산으로 부모-자식을 바로 찾을 수 있어서.

Q4. Up-heap과 Down-heap의 차이는?

> **힌트**

어느 연산 후에, 어느 방향으로 움직이는가.

> **정답**

개념은 힌트 참고.

정답: Up-heap은 Insert 후 루트 방향으로 거슬러 올라가며 회복, Down-heap은 Delete 후 리프 방향으로 내려가며 회복.

Q5. Heapify에서 루트가 아니라 마지막 비단말 노드부터 시작하는 이유는?

> **힌트**

Down-heap의 전제 조건: 자식 서브트리가 이미 힙이어야 한다. 리프는 그 자체로 힙.

> **정답**

개념은 힌트 참고.

① Down-heap은 자식들이 이미 올바른 힙일 때만 부모를 올바른 위치로 끌어내릴 수 있다.

② 리프 노드는 자식이 없으므로 그 자체로 힙이다.

③ 따라서 리프 바로 위의 마지막 비단말 노드(⌊n/2⌋)부터 시작해 루트까지 거슬러 올라가며 힙을 점진적으로 병합한다.

정답: 자식 서브트리가 먼저 힙이어야 Down-heap이 성립하므로, 가장 작은 서브트리부터 Bottom-up으로 처리해야 하기 때문.